



Islamic University of Technology

Department of Computer Science and Engineering (CSE)

CSE 4206: Digital Logic Design Lab
Summer, 2023-2024

Faisal Hussain, Assistant Professor, CSE
Aashnan Rahman, Jr. Lecturer, CSE

Lab 2

Implementation Using Universal Gates and Simplification of
Boolean Expressions

Contents

1	Introduction	2
2	Universal Gates	2
3	Implementing Basic Gates Using Universal Gates	2
3.1	Using Only NAND Gates	2
3.2	Using Only NOR Gates	3
4	Circuit Simplification	3
4.1	Operator Precedence	3
4.2	Boolean Algebra Laws	3
5	Overview of Logic Gates and ICs	4
6	IC Internal Diagram	4
7	Experiment 2: Implementation Using Universal Gates and Simplification of Boolean Expressions	5
7.1	Objectives	5
7.2	Tasks	5
7.2.1	Task 2	6
7.2.2	Task 2	7

1 Introduction

In the previous lab, you learned the basics of Digital Logic Design. You got familiar with the trainer kit, worked with logic gate ICs and built a simple combinational circuit.

In today's lab, we will focus on two things: universal gates and simplified circuits. You will see how NAND and NOR gates can be used to build any logic circuit. You'll also learn how to simplify logic expressions and build circuits using fewer gates.

By the end of the lab, you'll understand how to use universal gates and how to create simpler, more efficient circuits.

2 Universal Gates

Universal gates are logic gates that can be used to create any other type of logic gate—AND, OR, NOT, XOR, etc.—using just one type of gate. The two universal gates are -

- NAND (Not AND)
- NOR (Not OR)

These gates are called "universal" because they can be used to build entire digital circuits without needing any other type of gate.

This is very helpful in circuit design, especially in practical applications where minimizing the number of components is important. Using only one type of gate can reduce cost, save space, and simplify manufacturing.

In today's lab, you'll learn how to use only NAND or only NOR gates to build basic logic functions. This will help you understand how powerful these gates are and why they're so widely used in digital electronics.

3 Implementing Basic Gates Using Universal Gates

Basic logic gates such as AND, OR, and NOT form the foundation of all digital circuits. More complex circuits are built by combining these basic gates.

Universal gates, such as NAND and NOR, are powerful because they can be used to construct all basic gates. This means that if we can implement AND, OR, and NOT using only NAND or only NOR, then we can build any digital circuit using just one type of gate. This not only shows the versatility of universal gates but also makes them very useful in practical circuit design.

3.1 Using Only NAND Gates

- NOT Gate:

$$\overline{A} = \overline{(A \cdot A)}$$

- AND Gate:

$$A \cdot B = \overline{\overline{(A \cdot B)}}$$

- OR Gate: (Using De Morgan's Law)

$$A + B = \overline{\overline{A + B}} = \overline{\overline{A} \cdot \overline{B}}$$

3.2 Using Only NOR Gates

- NOT Gate:

$$\overline{A} = \overline{(A + A)}$$

- OR Gate:

$$A + B = \overline{\overline{A + B}}$$

- AND Gate: (Using De Morgan's Law)

$$A \cdot B = \overline{\overline{A \cdot B}} = \overline{\overline{A} + \overline{B}}$$

4 Circuit Simplification

In digital electronics, circuit simplification is the process of reducing the number of gates and connections needed to implement a logic function. This helps minimize hardware cost, power consumption, and increases the speed of the circuit. Simplification is done by applying Boolean algebra rules to reduce expressions to their simplest forms without changing their function.

4.1 Operator Precedence

When evaluating Boolean expressions, the following operator precedence is followed (from highest to lowest):

1. Parenthesis
2. NOT ($\overline{A}$)
3. AND ($A \cdot B$)
4. OR ($A + B$)

Parentheses should be used to make expressions clear and to override default precedence when needed.

4.2 Boolean Algebra Laws

Law	Expressions	
Identity	$A + 0 = A$	$A \cdot 1 = A$
Null / Domination	$A + 1 = 1$	$A \cdot 0 = 0$
Idempotent	$A + A = A$	$A \cdot A = A$
Inverse	$A + \overline{A} = 1$	$A \cdot \overline{A} = 0$
Double Negation	$\overline{\overline{A}} = A$	
Commutative	$A + B = B + A$	$A \cdot B = B \cdot A$
Associative	$A + (B + C) = (A + B) + C$	$A \cdot (B \cdot C) = (A \cdot B) \cdot C$
Distributive	$A \cdot (B + C) = A \cdot B + A \cdot C$	$A + B \cdot C = (A + B) \cdot (A + C)$
Absorption	$A + A \cdot B = A$	$A \cdot (A + B) = A$
De Morgan's Theorem	$\overline{A \cdot B} = \overline{A} + \overline{B}$	$\overline{A + B} = \overline{A} \cdot \overline{B}$

5 Overview of Logic Gates and ICs

Gate Name	Logical Expression	IC Number
AND	$A \cdot B$	7408
OR	$A + B$	7432
NOT	$\overline{A}$	7404
NAND	$\overline{A \cdot B}$	7400
NOR	$\overline{A + B}$	7402
XOR	$A \oplus B$	7486
XNOR	$\overline{A \oplus B}$	74266

Table 1: Logic Gates and their corresponding ICs (TTL – 74 series)

6 IC Internal Diagram

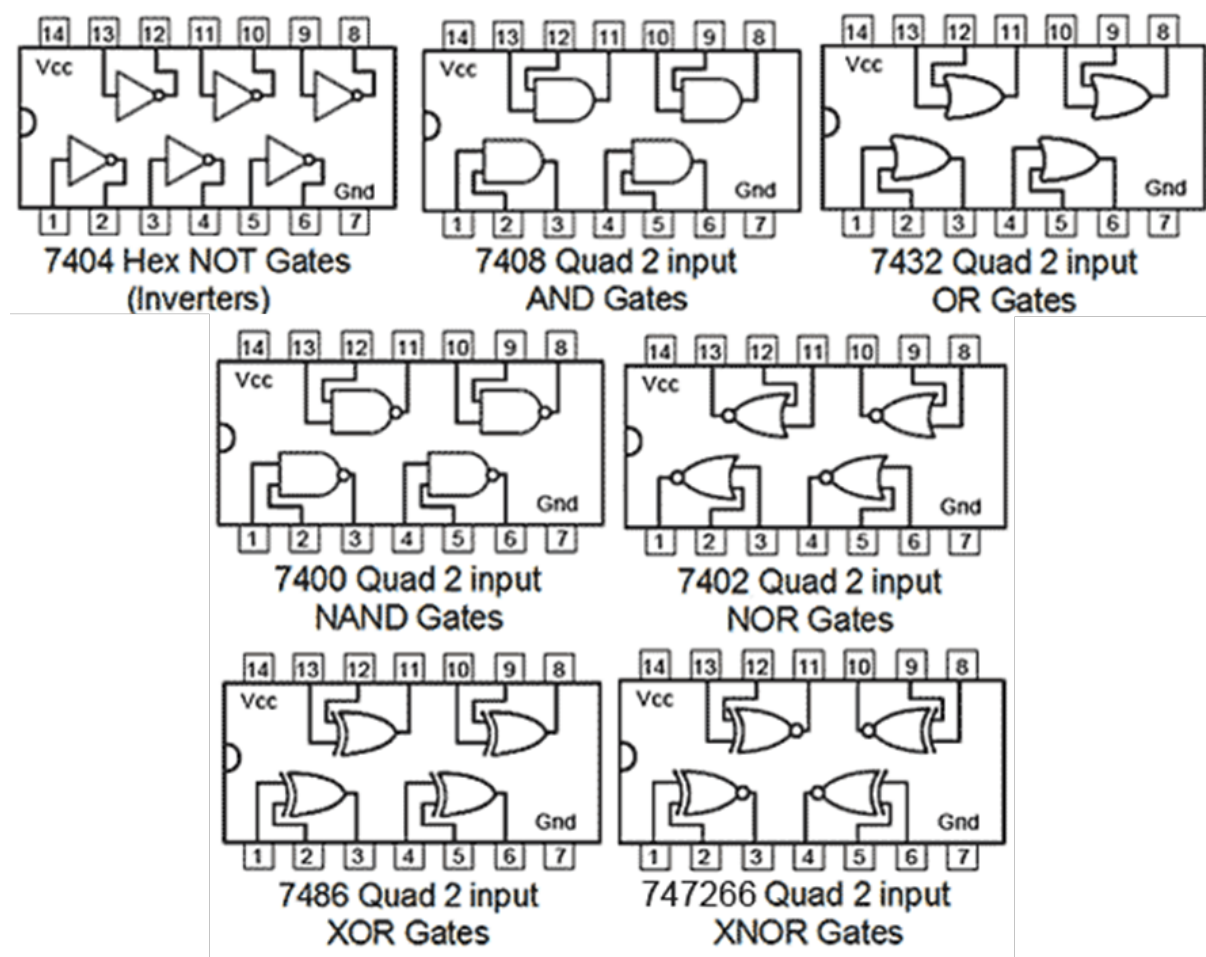


Figure 1: Internal Structure of ICs

7 Experiment 2: Implementation Using Universal Gates and Simplification of Boolean Expressions

7.1 Objectives

The objectives of this experiment are as follows:

1. Understand the concept of universal gates (NAND and NOR) and their significance in digital logic design.
2. Implement other logic gates (AND, OR, NOT, NAND, NOR, XOR) using only NAND or only NOR gates.
3. Apply Boolean algebra laws to simplify logical expressions.
4. Realize simplified Boolean expressions using logic gate ICs.
5. Construct and test simplified logic circuits on the digital trainer kit.
6. Analyze the difference between simplified and unsimplified circuits in terms of gate count and efficiency.

7.2 Tasks

Today's lab task are as follows -

7.2.1 Task 2

Complete the following table using only **NAND** gate.

Gate	Simplification	Simplified Circuit	Truth Table		
AND			A	B	Output
			0	0	
			0	1	
			1	0	
			1	1	
OR			A	B	Output
			0	0	
			0	1	
			1	0	
			1	1	
NOT			A	Output	
			0		
			1		
NAND			A	B	Output
			0	0	
			0	1	
			1	0	
			1	1	
NOR			A	B	Output
			0	0	
			0	1	
			1	0	
			1	1	
XOR			A	B	Output
			0	0	
			0	1	
			1	0	
			1	1	

Table 2: Task 2

7.2.2 Task 2

Given,

$$F_1 = ((A + \overline{B}) \cdot \overline{(B + C)}) + (\overline{A} \cdot \overline{C} \cdot B)$$

Your task is to do the following -

- Simplify the Boolean expression F_1 .
- Implement the circuit diagram for the simplified expression (F_2).
- Implement the simplified circuit using only **NAND** gates (F_3).
- Implement the following on the same breadboard:
 - The original (unsimplified) circuit using basic gates.
 - The simplified circuit using basic gates.
 - The simplified circuit using only **NAND** gates.

Connect the outputs of all three circuits to three separate LEDs and verify that all outputs are identical.

- Complete the truth table to confirm the correctness of each implementation.

Simplify the expression

$$F_1 = ((A + \overline{B}) \cdot \overline{(B + C)}) + (\overline{A} \cdot \overline{C} \cdot B)$$

Simplify F_1	Express F_2 using NAND
Simplified expression, $F_2 =$	$F_3 =$

Circuit Diagram for F_1	Circuit Diagram for F_2
Circuit Diagram for F_3	

Truth Table

A	B	C	F ₁	F ₂	F ₃
0	0	0			
0	0	1			
0	1	0			
0	1	1			
1	0	0			
1	0	1			
1	1	0			
1	1	1			

Table 3: Proof by Truth Table

	F ₁	F ₂	F ₃
Term			
Literal Count			
ICs Used			
Gates Used			

Table 4: Comparison of Original (F₁), Simplified (F₂), NAND-based (F₃) circuit